

THE INFLUENCE OF ARTIFICIAL INTELLIGENCE ON E-GOVERNANCE AND CYBER SECURITY IN SMART CITIES

A Mini Project report submitted in partial fulfillment of the requirements for the award of the degree
of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

by

R. AKSHITHA	(22S41A05A8)
R. AKSHAYA	(23S45A0508)
V. CHARAN	(22S41A05C6)
P. VENUGOPALA KRISHNA	(22S41A05A2)

Under the Guidance of
Mrs. A. SARADA
Assistant Professor



Department of Computer Science & Engineering
VAAGESWARI COLLEGE OF ENGINEERING
Accredited by NAAC with A+ Grade
(Affiliated to JNTU Hyderabad & Approved by AICTE, New Delhi)
Ramakrishna colony, Karimnagar-505527.
May-2025

Department of Computer Science & Engineering
VAAGESWARI COLLEGE OF ENGINEERING
Accredited by NAAC with A+ Grade
(Affiliated to JNTU Hyderabad & Approved by AICTE, New Delhi)
Ramakrishna colony, Karimnagar-505527.



CERTIFICATE

This is to certify that the project report entitled “**THE INFLUENCE OF ARTIFICIAL INTELLIGENCE ON E-GOVERNANCE AND CYBER SECURITY IN SMART CITIES**” submitted by the following students in partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology in CSE and is a Bonafide record of the work performed by.

R. AKSHITHA	(22S41A05A8)
R. AKSHAYA	(23S45A0508)
V. CHARAN	(22S41A05C6)
P. VENUGOPALA KRISHNA	(22S41A05A2)

Internal Guide

Mrs. A. SARADA
Assistant Professor

Head of the department

Dr. N. CHANDRAMUOLI
Professor

Principal

Dr. CH. SRINIVAS

External Examiner

DECLARATION

We hereby declare that the project titled “**THE INFLUENCE OF ARTIFICIAL INTELLIGENCE ON E-GOVERNANCE AND CYBER SECURITY IN SMART CITIES**” submitted to Vaageswari College of Engineering, affiliated to Jawaharlal Nehru Technological University Hyderabad (JNTUH) for the award of the Degree of Bachelor of technology in CSE is a result of original research carried-out in this work. It is further declared that the report or any part thereof has not been previously submitted to any University or Institute for the award of degree or diploma.

1. Name of the Student: R. AKSHITHA

Hall Ticket Number:22S41A05A8

2. Name of the Student: R. AKSHAYA

Hall Ticket Number:23S45A0508

3. Name of the Student: V. CHARAN

Hall Ticket Number:22S41A05C6

4. Name of the student: P. VENUGOPALA KRISHNA

Hall Ticket Number:22S41A05A2

Date:

ACKNOWLEDGEMENT

We wish to pay our sincere thanks to **Dr. CH. SRINIVAS**, Principal, Vaageswari College of Engineering, Karimnagar, for providing all required facilities and his support during the Project Work.

We would like to thank **Dr. N. CHANDRAMOULI**, Professor and HOD of the Computer Science and Engineering Department for his valuable suggestions during the Project Work.

We sincerely extend our thanks to the mini project guide **Mrs. A. SARADA** Assistant Professor of Computer Science and Engineering Department, for sparing his valuable in guiding the Minor project work and giving feedback with many useful suggestions during the minor project work.

We are also conveying our heartfelt thanks to the Institute authority, Department, Library, and Laboratory staff of Vaageswari College of Engineering for their co-operation during our Project Work. We thank our beloved friends for their help and encouragement regarding the concepts and presentation.

R. AKSHITHA	(22S41A05A8)
R. AKSHAYA	(23S45A0508)
V. CHARAN	(22S41A05C6)
P. VENUGOPALA KRISHNA	(22S41A05A2)

ABSTRACT

Artificial intelligence (AI) has been identified as a critical technology of Fourth Industrial Revolution (Industry 4.0) for protecting computer network systems against cyber-attacks, malware, phishing, damage, or illicit access. AI has potential in strengthening the cyber capabilities and safety of nationstates, local governments, and non-state entities through e-Governance. Existing research provides a mixedassociation between AI, e-Governance, and cybersecurity; however, this relationship is believed to be context-specific. AI, e-Governance, and cybersecurity influence and are affected by various stakeholders possessing a variety of knowledge and expertise in respective areas. To fill this context specific gap, this study investigates the direct relationship between AI, eGovernance, and cybersecurity. Furthermore, this study examines the mediating role of e-Governance between AI and cybersecurity and moderating effect of stakeholders involvement on the relationship between AI, e-Governance, and cybersecurity. The results of PLS-SEM path modeling analysis revealed a partial mediating impact of e-Governance between AI and cybersecurity. Likewise, moderating influence of stakeholders involvement was discovered on the relationship between AI and e-Governance, as well as between e-Governance and cybersecurity. It implies that stakeholders involvement has vital significance in AI and e-Governance because all stakeholders have interest in vibrant, transparent, and secured cyberspace while using e-services. This study provides practical implications for governmental bodies of smart cities for strengthening their cybersecurity measures.

LIST OF FIGURES

S.NO	TABLES/FIGURES	PAGE NO'S
1	System Architecture	15
2	UML Diagrams	16-24
3	4.1.1 Use Case Diagram	17
4	4.1.2. Class Diagram	18
5	4.1.3 Sequence Diagram	19
6	4.1.4 Collaboration Diagram	20
7	4.1.5 Activity Diagram	21
8	4.1.6 Component Diagram	22
9	4.1.7 Deployment Diagram	22
10	4.1.8 ER Diagram	23
11	4.19 Data Dictionary/Data Set	23-24
12	About Python	28-65
13	Screenshots	84-86

INDEX

TOPICS	Page No's
➤ Certificates	
➤ Declaration	
➤ Acknowledgement	
➤ Abstract	
➤ Figures/Tables	
CHAPTER-1: INTRODUCTION	
CHAPTER-2: LITERATURE SURVEY	
CHAPTER-3: REQUIREMENT ANALYSIS	
3.1 Existing System	
3.2 Proposed System	
3.3 System Requirements	
3.4 Feasibility Study	
CHAPTER-4: SYSTEM DESIGN	
4.1 System Architecture	
4.2 UML Diagrams	
CHAPTER-5: IMPLEMENTATION	
5.1 Modules	
5.2 Technology	
5.3 System Testing	
CHAPTER-6: RESULTS/DISCUSSIONS	
CHAPTER-7: CONCLUSION	
REFERENCES	

CHAPTER-1

INTRODUCTION

Cyber security has become a critical and vital topic that requires protecting the computer network from potential threats in today's modern world. A cyber-attack is a deliberate attack targeting computer networks, relevant data, programs, and electronic information, resulting in sub-national entities inciting violence towards noncombatant opponents. As technology develops, so do cyber threats, necessitating the development of new prevention strategies. It has been alleged that cyber-attacks have become more prevalent in the industrial sector, resulting in serious infrastructure damage and significant monetary loss. The rise of cyber-attacks among organizations is primarily due to the growing reliance on online technologies that enable the storage of personal and economic data .

Artificial intelligence (AI) applications can positively influence the cyber capabilities and national security of the sovereign nation, regional government entities, and non-state organizations. AI is a reliable technique for mitigating cyber-attack effects. AI is machine intelligence that executes activities connected with intelligence. Human professionals' expertise is integrated for strategic planning and decision-making, including making medical diagnoses and getting insights from expertise in concluding. In terms of cyber security, Zarina et al., have illustrated that AI has both beneficial and harmful effects, with the harmful effect of facilitating the instigation phase of cyber attacks, resulting in quicker and more devastating attacks. Looking forward, AI has the potential to greatly improve cyber security by increasing security precautions and promoting security in cyberspace. Furthermore, AI assists security experts in detecting cyber hazard symptoms and has enhanced the machine learning applications for malware classification and networked intrusion detection. Lastly, the modern phenomenon in AI has transformed innovative solutions and improved city external attacks against serious security threats .

A smart city provides multiple innovative solutions to several challenges that city administration faces. However, information and communication technology (ICT) has become a vital component of e-Government. Implementing ICT into a city's infrastructure introduces hazards and obstructions. People frequently use insecure Wi-Fi networks to check their email messages, ebanking, and other digital services, uncovering themselves to cybercrimes including hacking, denials of service, and cracking. Cyber security applying technologies to protect e-Government services is among the most important distinctive features that can be utilized to categorize safe cities globally. Somewhere in this tendency, the 'inclusive smart city' framework has triggered strong interest because it emphasizes the importance of interpersonal and social capital in urban initiatives that focus on stakeholders' inclusion in the Digital Realm and involving inhabitants in service improvement to implement appropriate government services that match citizens' necessities . Recent studies on e-services and technologies also have emphasized the importance of implementing a citizenscentered strategy for smart cities because it is expected to develop strong social ecologies that depend

strongly on web technology. Consequently, web technologies and services can significantly impact stakeholder interactions .

Although previous literature demonstrated influence of AI in smart mobility , energy management, public services, climate change, and smart security in smart cities, cyber security has widely been neglected, especially in the context of stakeholders who use online government services. To fill this contextual gap, this study formulated the following research question:

- How AI applications used in smart cities influence cyber security directly?
- How AI applications used in smart cities influence e-Governance and e-Governance impacts cyber security directly?
- Does e-Governance play a mediating role between the relationship of AI applications and cyber security?
- Additionally, this study examines the moderating role of stakeholders' involvement in the relationship between AI and e-Governance and on the relationship between e-Governance and cyber security.

These main research questions are attempted to address empirically in this study, based on the premise that the interactions are context-dependent. Figure 1 explains the channel of the study's proposed framework to classify cyber security level in a smart city. The moderating significance of stakeholder involvement was systematically examined by using structural equational modeling (SEM) in Smart PLS 4.0. PLS-SEM path modeling was selected as the analytical tool because of its widespread utilization in examining research frameworks in prior studies and its acknowledged appropriateness for analyzing complex research models. Section II proceeds with a literature background on the relationships between artificial intelligence, e-Governance, stakeholder involvement, cyber security, and the key hypotheses under consideration. The data sampling, research framework, methodology, and analysis are described in Section III. The statistical findings are presented in Section IV. Section V summarizes the discussions, draws conclusions, and recommends future research.

CHAPTER-2

LITERATURE SURVEY

TITLE: The Influence of Artificial Intelligence on E-Governance and Cybersecurity in Smart

Cities: A Stakeholder's Perspective

AUTHOR: Syed Asad Abbas Bokhari

ABSTRACT: Artificial intelligence (AI) has been identified as a critical technology of Fourth Industrial Revolution (Industry 4.0) for protecting computer network systems against cyberattacks, malware, phishing, damage, or illicit access. AI has potential in strengthening the cyber capabilities and safety of nation-states, local governments, and non-state entities through eGovernance. Existing research provides a mixed association between AI, e-Governance, and cybersecurity; however, this relationship is believed to be context-specific. AI, e-Governance, and cybersecurity influence and are affected by various stakeholders possessing a variety of knowledge and expertise in respective areas. To fill this context specific gap, this study investigates the direct relationship between AI, e-Governance, and cybersecurity. Furthermore, this study examines the mediating role of e-Governance between AI and cybersecurity and moderating effect of stakeholders involvement on the relationship between AI, e-Governance, and cybersecurity. The results of PLS-SEM path modeling analysis revealed a partial mediating impact of e-Governance between AI and cybersecurity. Likewise, moderating influence of stakeholders involvement was discovered on the relationship between AI and e-Governance, as well as between e-Governance and cybersecurity. It implies that stakeholders involvement has vital significance in AI and eGovernance because all stakeholders have interest in vibrant, transparent, and secured cyberspace while using e-services. This study provides practical implications for governmental bodies of smart cities for strengthening their cybersecurity measures.

TITLE: The Influence of Artificial Intelligence on E-Governance and Cybersecurity in Smart Cities

AUTHOR: Seunghwan Myeong

ABSTRACT: AI has potential in strengthening the cyber capabilities and safety of nationstates, local governments, and non-state entities through e-Governance. Existing research provides a mixed association between AI, e-Governance, and cybersecurity; however, this relationship is believed to be context-specific. AI, e-Governance, and cybersecurity influence and are affected by various stakeholders possessing a variety of knowledge and expertise in respective areas. To fill this context specific gap, this study investigates the direct relationship between AI, e-Governance, and cybersecurity. Furthermore, this study examines the mediating role of e-Governance between AI and cybersecurity and moderating effect of stakeholders involvement on the relationship between AI, e-Governance, and cybersecurity. The results

of PLS-SEM path modeling analysis revealed a partial mediating impact of e-Governance between AI and cybersecurity

TITLE: Artificial Intelligence in the Smart City — A Literature Review

AUTHOR: Danuta Szpilko

ABSTRACT: The influence of artificial intelligence (AI) in smart cities has resulted in enhanced efficiency, accessibility, and improved quality of life. However, this integration has brought forth new challenges, particularly concerning data security and privacy due to the widespread use of Internet of Things (IoT) technologies. The article aims to provide a classification of scientific research relating to artificial intelligence in smart city issues and to identify emerging directions of future research. A systematic literature review based on bibliometric analysis of Scopus and Web of Science databases was conducted for the study. Research query included TITLE-ABS-KEY

(“smart city” AND “artificial intelligence”) in the case of Scopus and TS = (“smart city” AND “artificial intelligence”) in the case of the Web of Sciences database. For the purpose of the analysis, 3101 publication records were qualified. Based on bibliometric analysis, seven research areas were identified: safety, living, energy, mobility, health, pollution, and industry. Urban mobility has seen significant innovations through AI applications, such as autonomous vehicles (AVs), electric vehicles (EVs), and unmanned aerial vehicles (UAVs), yet security concerns persist, necessitating further research in this area. AI’s impact extends to energy management and sustainability practices, demanding standardised regulations to guide future research in renewable energy adoption and developing integrated local energy systems. Additionally, AI’s applications in health, environmental management, and the industrial sector require further investigation to address data handling, privacy, security, and societal implications, ensuring responsible and sustainable digitisation in smart cities.

TITLE: Artificial intelligence in the smart city

AUTHOR: Danuta Szpilko

ABSTRACT: The influence of artificial intelligence (AI) in smart cities has resulted in enhanced efficiency, accessibility, and improved quality of life. However, this integration has brought forth new challenges, particularly concerning data security and privacy due to the widespread use of Internet of Things (IoT) technologies. The article aims to provide a classification of scientific research relating to artificial intelligence in smart city issues and to identify emerging directions of future research. A systematic literature review based on bibliometric analysis of Scopus and Web of Science databases was conducted for the study. Research query included TITLE-ABS-KEY

(“smart city” AND “artificial intelligence”) in the case of Scopus and TS = (“smart city” AND “artificial intelligence”) in the case of the Web of Sciences database. For the purpose of the analysis, 3101 publication records were qualified. Based on bibliometric analysis, seven research areas were identified:

safety, living, energy, mobility, health, pollution, and industry. Urban mobility has seen significant innovations through AI applications, such as autonomous vehicles (AVs), electric vehicles (EVs), and unmanned aerial vehicles (UAVs), yet security concerns persist, necessitating further research in this area. AI's impact extends to energy management and sustainability practices, demanding standardised regulations to guide future research in renewable energy adoption and developing integrated local energy systems. Additionally, AI's applications in health, environmental management, and the industrial sector require further investigation to address data handling, privacy, security, and societal implications, ensuring responsible and sustainable digitisation in smart cities.

TITLE: Security, Privacy and Risks Within Smart Cities

AUTHOR: Elvira Ismagilova, Laurie Hughes

ABSTRACT: The complex and interdependent nature of smart cities raises significant political, technical, and socioeconomic challenges for designers, integrators and organisations involved in administering these new entities. An increasing number of studies focus on the security, privacy and risks within smart cities, highlighting the threats relating to information security and challenges for smart city infrastructure in the management and processing of personal data. This study analyses many of these challenges, offers a valuable synthesis of the relevant key literature, and develops a smart city interaction framework. The study is organised around a number of key themes within smart cities research: privacy and security of mobile devices and services; smart city infrastructure, power systems, healthcare, frameworks, algorithms and protocols to improve security and privacy, operational threats for smart cities, use and adoption of smart services by citizens, use of blockchain and use of social media. This comprehensive review provides a useful perspective on many of the key issues and offers key direction for future studies. The findings of this study can provide an informative research framework and reference point for academics and practitioners.

CHAPTER-3

REQUIREMENT ANALYSIS

3.1 EXISTING SYSTEM

Smart city is a captivating concept characterized by its intelligent features. Its scope extends beyond improving the level of urban economic efficiency and the reduction of costs and resource consumption. Rather, it encompasses the integration of different components of the city through intelligent gadgets and the application of digital technologies or information and communication technology (ICT) to enhance service delivery. The transformation of conventional urban areas into smart cities has resulted in a higher living standard for citizens .

An illustration of a smart city can be outlined by using several fundamental elements, as exemplified in Figure 2. Smart government comprises various aspects such as smart office, smart supervision, smart services, and smart decision-making to enhance the performance of city governance and optimize the life standard of citizens by establishing a bilateral collaboration between the government and citizens . Smart public services offer various electronic information and online services to enhance the standard of living and satisfaction of the public, thereby developing the perception of a service-oriented government. The evolution of a smart economy can facilitate the smooth development of resource driven cities, enhance the efficiency of urban economies, and generate sustainable employment opportunities .

Smart healthcare systems that utilize e-health records to forecast the individual's health, like remote tracking of individuals with cardiac disease, has the potential to assess the state of vulnerability and furnish essential information for optimal treatment . Smart education is a concept that involves using data-centric intelligent education in different contexts in smart cities to deliver individuals a smooth educational experience with customized individual assistance . Smart buildings that effectively apply different information. The building is capable of satisfying the necessities of its users and residents, as well as identifying any defects in its operation. Buildings with features such as security, flexibility, ease of use, and efficiency are extremely attractive. Smart transport systems are multifaceted and digitally managed to help with urban development and decisionmaking, thereby organizing

smart transportation. Strategic travel scheduling can be achieved by the use of route projection and real-time roadway state monitoring . Smart Security offers an assortment of benefits including detection, alarm, emergency assistance, and other functions pertaining to personal protection of individuals and safeguarding cybersecurity].

It is well-established that various infrastructure systems, including energies, grid system, healthcare, traffic, transportation, water distribution, and wastewater disposal, are furnished with computer networks. The use of Internet of Things has resulted in the emergence of smart cities, which aim at improving their facilities and developing more sophisticated, effective, and eco-friendly solutions. Nonetheless, a study ABI

Research has projected that by 2024, barely 44% of the overall cybersecurity expenses for critical systems will be assigned to sectors such as healthcare, security, water, transport, and other related areas, leading to a significant lacking funding for protecting infrastructure against cybersecurity risks . Consequently, there is a likelihood of various challenges involving cyberattacks on crucial urban infrastructure, resulting in serious repercussions including the act of hijacking infrastructure communication and encrypting malware to disable computer systems has the potential to significantly impact the financial security of a city, resulting in substantial losses to both the finances and assets of inhabitants. Similarly, the disruption or destruction of communication systems, power grids, water conservation mechanisms, and other facilities can destroy the social system and cause an outbreak of a state of anxiety. Moreover, interfering with sensor data for creating a situation of chaos, such as in disaster detection technologies, and stealing of crucial information such as people, healthcare, customers, and private information.

Several prior research has explored the significance of artificial intelligence in detecting and preventing cyberattacks , combating terrorism , enhancing security in strategic sectors , and building resilience in vulnerable sovereign places . Soni stated in his study that Information obtained from a broad selection of scientific and engineering specialists suggests that AI development depends on the United States capabilities to reconcile the advantages and disadvantages of AI, specifically in cybersecurity. AI is universally perceived among the most impressive technologies of the digital world, and cybersecurity is undoubtedly the domain that might benefit greatly from it. Optimization algorithms, strategies, devices, and companies providing AI-based solutions are evolving in international security markets It is emphasized that privacy and public security constitute critical concerns in smart cities which require additional legislative, technological, and administrative attention. Combating cybercrime in smart cities is essential for making this technology as advantageous and credible as possible for community acceptance. All stakeholders, particularly legislators, administrations, judicial systems, power companies, telecom firms, automobile manufacturers, cloud hosting, research institutes, and industries, will have to continue their assistance and endeavors .

DISADVANTAGES

- The complexity of data: Most of the existing machine learning models must be able to accurately interpret large and complex datasets to detect Cybersecurity.
- Data availability: Most machine learning models require large amounts of data to create accurate predictions. If data is unavailable in sufficient quantities, then model accuracy may suffer.
- Incorrect labeling: The existing machine learning models are only as accurate as the data trained using the input dataset. If the data has been incorrectly labeled, the model cannot make accurate predictions.

3.2 PROPOSED SYSTEM

The primary objective of the proposed system is to investigate the relationship between artificial intelligence and cybersecurity, performing e-Governance as a mediator and stakeholders' involvement as a moderator. A longitudinal research method is conducted to investigate the hypothesis derived from this study and

ascertain the findings. It comprises a study into perceptions of the importance of AI in cybersecurity in smart cities. The primary data for this study was collected from 478 respondents through a survey questionnaire distributed via emails and online through several social media networks.

Respondents were adequately explained about answers and were encouraged to respond to the questionnaire with utmost honesty, that may minimize issues about potential bias. Lastly, participants might opt out of the survey at any moment.

ADVANTAGES

- Artificial intelligence applications in smartcities contribute to e-Governance positively.
- E-Governance execution in smart cities affect cybersecurity positively.
- E-Governance mediates between artificial intelligence and cybersecurity positively.

3.3 SYSTEM REQUIREMENTS

FUNCTIONAL REQUIREMENTS

• Network Follower

Network Follower as the user he register and login into the application and performs their operations

• Maintainer

Maintainer provides services to the network follower, he takes care of the application

NON FUNCTIONAL REQUIREMENTS

HARDWARE REQUIREMENTS

- Processor - Pentium –IV
- RAM - 4 GB (min)
- Hard Disk - 20 GB
- Key Board - Standard Windows Keyboard
- Mouse - Two or Three Button Mouse
- Monitor - SVGA

SOFTWARE REQUIREMENTS:

- ✚ Operating system : Windows 7 Ultimate.
- ✚ Coding Language : Python.
- ✚ Front-End : Python.
- ✚ Back-End : Django-ORM
- ✚ Designing : Html, css, javascript.
- ✚ Data Base : MySQL (WAMP Server).

3.4 FEASIBILITY STUDY

The feasibility of the project is analyzed in this phase and business proposal is put forth with a very general plan for the project and some cost estimates. During system analysis the feasibility study of the proposed system is to be carried out. This is to ensure that the proposed system is not a burden to the company. For feasibility analysis, some understanding of the major requirements for the system is essential.

FEASIBILITY ANALYSIS

Three key considerations involved in the feasibility analysis are

- ECONOMICAL FEASIBILITY
- TECHNICAL FEASIBILITY
- SOCIAL FEASIBILITY

ECONOMICAL FEASIBILITY

This study is carried out to check the economic impact that the system will have on the organization. The amount of fund that the company can pour into the research and development of the system is limited. The expenditures must be justified. Thus the developed system as well within the budget and this was achieved because most of the technologies used are freely available. Only the customized products had to be purchased.

TECHNICAL FEASIBILITY

This study is carried out to check the technical feasibility, that is, the technical requirements of the system. Any system developed must not have a high demand on the available technical resources. This will lead to high demands on the available technical resources. This will lead to high demands being placed on the client. The developed system must have a modest requirement, as only minimal or null changes are required for implementing this system.

SOCIAL FEASIBILITY

The aspect of study is to check the level of acceptance of the system by the user. This includes the process of training the user to use the system efficiently. The user must not feel threatened by the system, instead must accept it as a necessity. The level of acceptance by the users solely depends on the methods that are employed to educate the user about the system and to make him familiar with it. His level of confidence must be raised so that he is also able to make some constructive criticism, which is welcomed, as he is the final user of the system.

CHAPTER-4

SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE

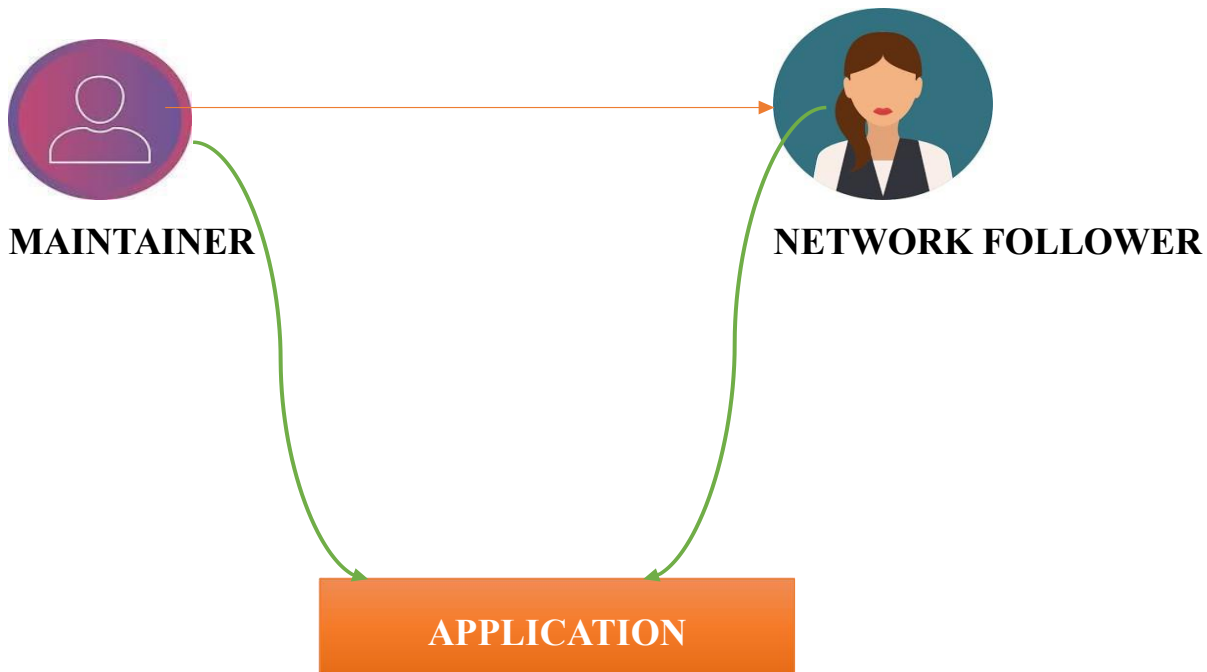


Fig 4.1: System Architecture

4.2 UML DIAGRAMS

UML stands for Unified Modeling Language. UML is a standardized general-purpose modeling language in the field of object-oriented software engineering. The standard is managed, and was created by, the Object Management Group. The goal is for UML to become a common language for creating models of object oriented computer software. In its current form UML is comprised of two major components: a Meta-model and a notation. In the future, some form of method or process may also be added to; or associated with, UML. The Unified Modeling Language is a standard language for specifying, Visualization, Constructing and documenting the artifacts of software system, as well as for business modeling and other nonsoftware systems. The UML represents a collection of best engineering practices that have proven successful in the modeling of large and complex systems. The UML is a very important part of developing objects oriented software and the software development process. The UML uses mostly graphical notations to express the design of software projects.

GOALS:

The Primary goals in the design of the UML are as follows:

1. Provide users a ready-to-use, expressive visual modeling Language so that they can develop and exchange meaningful models.
2. Provide extensibility and specialization mechanisms to extend the core concepts.

3. Be independent of particular programming languages and development process.
4. Provide a formal basis for understanding the modeling language.
5. Encourage the growth of OO tools market.
6. Support higher level development concepts such as collaborations, frameworks, patterns and components.
7. Integrate best practices.

4.2.1 USE CASE DIAGRAM:

A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.

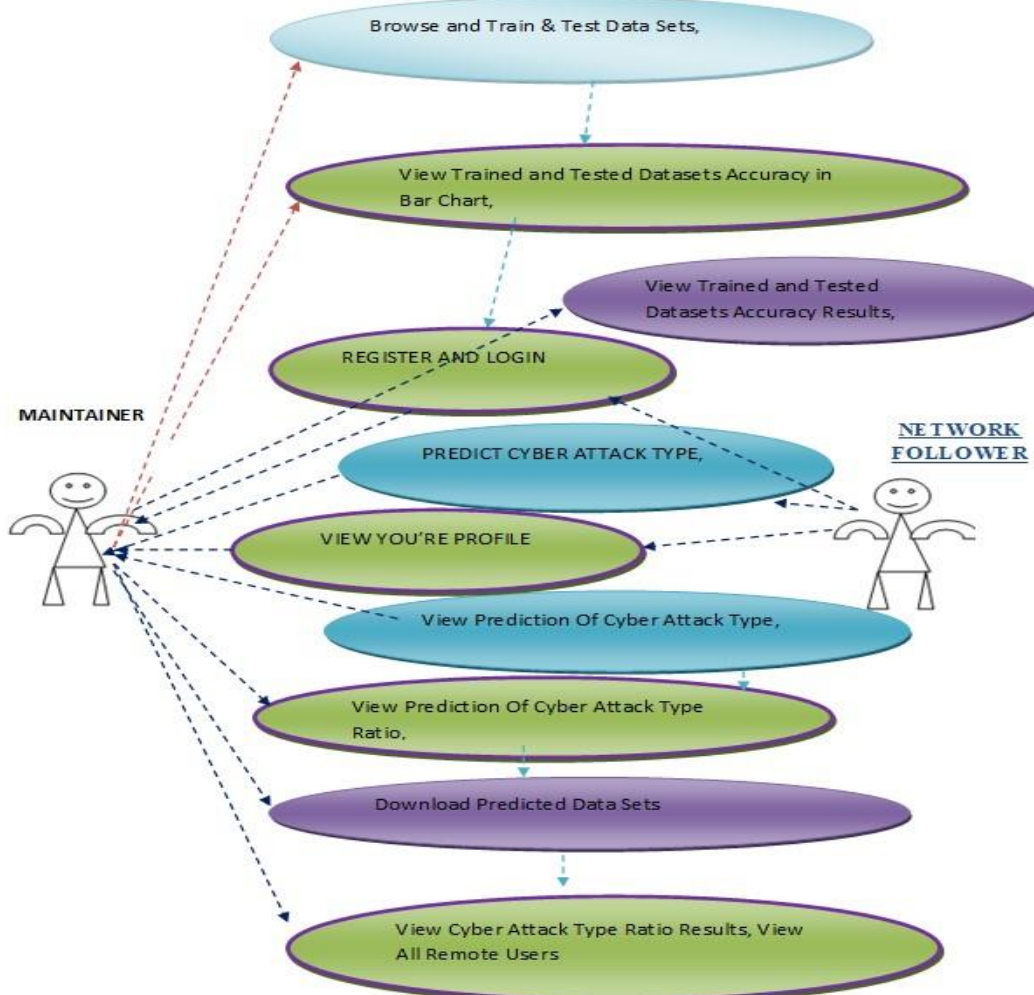


Fig 4.2.1: Use Case Diagram

4.2.2 CLASS DIAGRAM

In software engineering, a class diagram in the Unified Modeling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. It explains which class contains information.

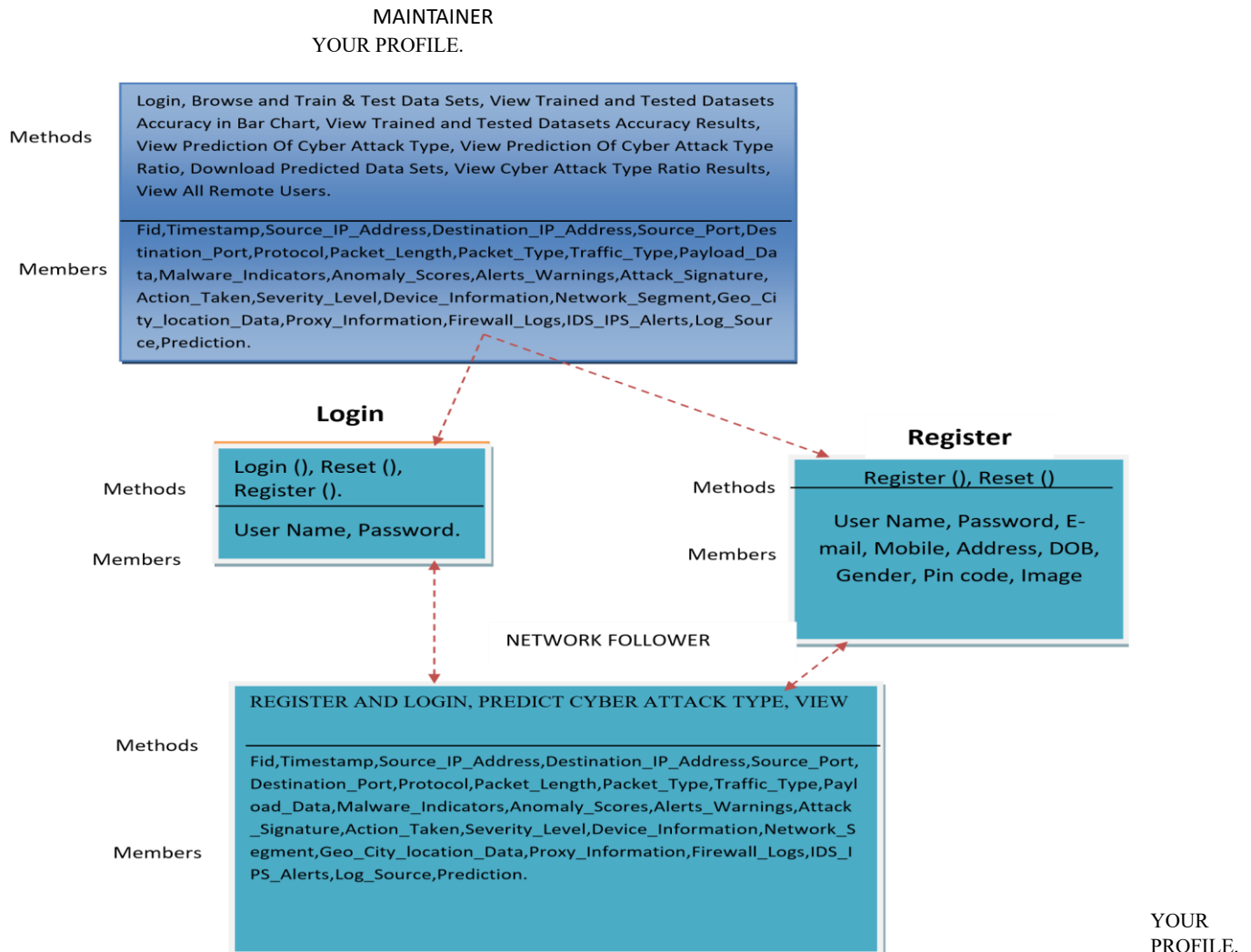


Fig 4.2.2: Class Diagram

4.2.3 SEQUENCE DIAGRAM:

A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams.

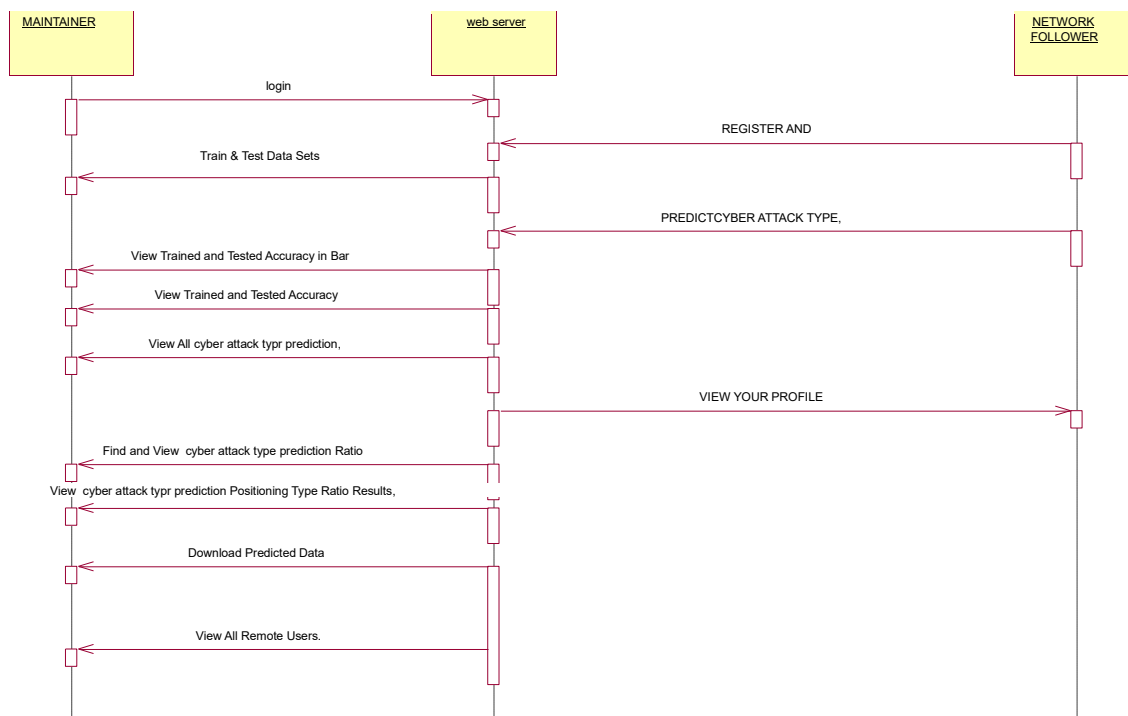


Fig 4.2.3: Sequence Diagram

4.2.4 COLLABORATION DIAGRAM

Activity diagrams are graphical representations of workflows of stepwise activities and actions with support for choice, iteration and concurrency. In the Unified Modeling Language, activity diagrams can be used to describe the business and operational step-by-step workflows of components in a system. An activity diagram shows the overall flow of control.

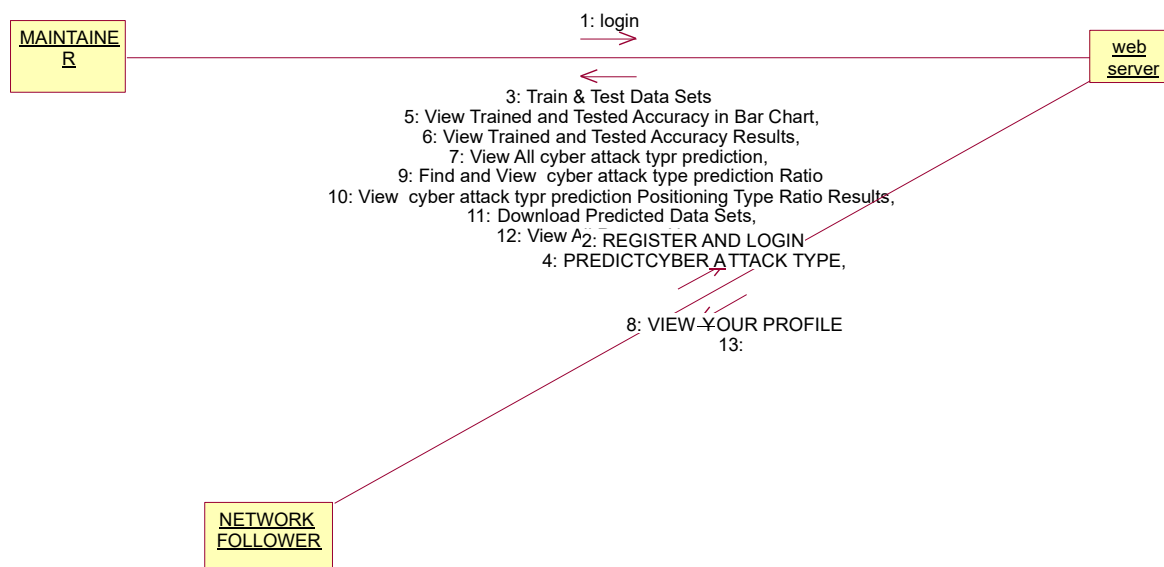


Fig 4.2.4: Collaboration Diagram

4.2.5 ACTIVITY DIAGRAM

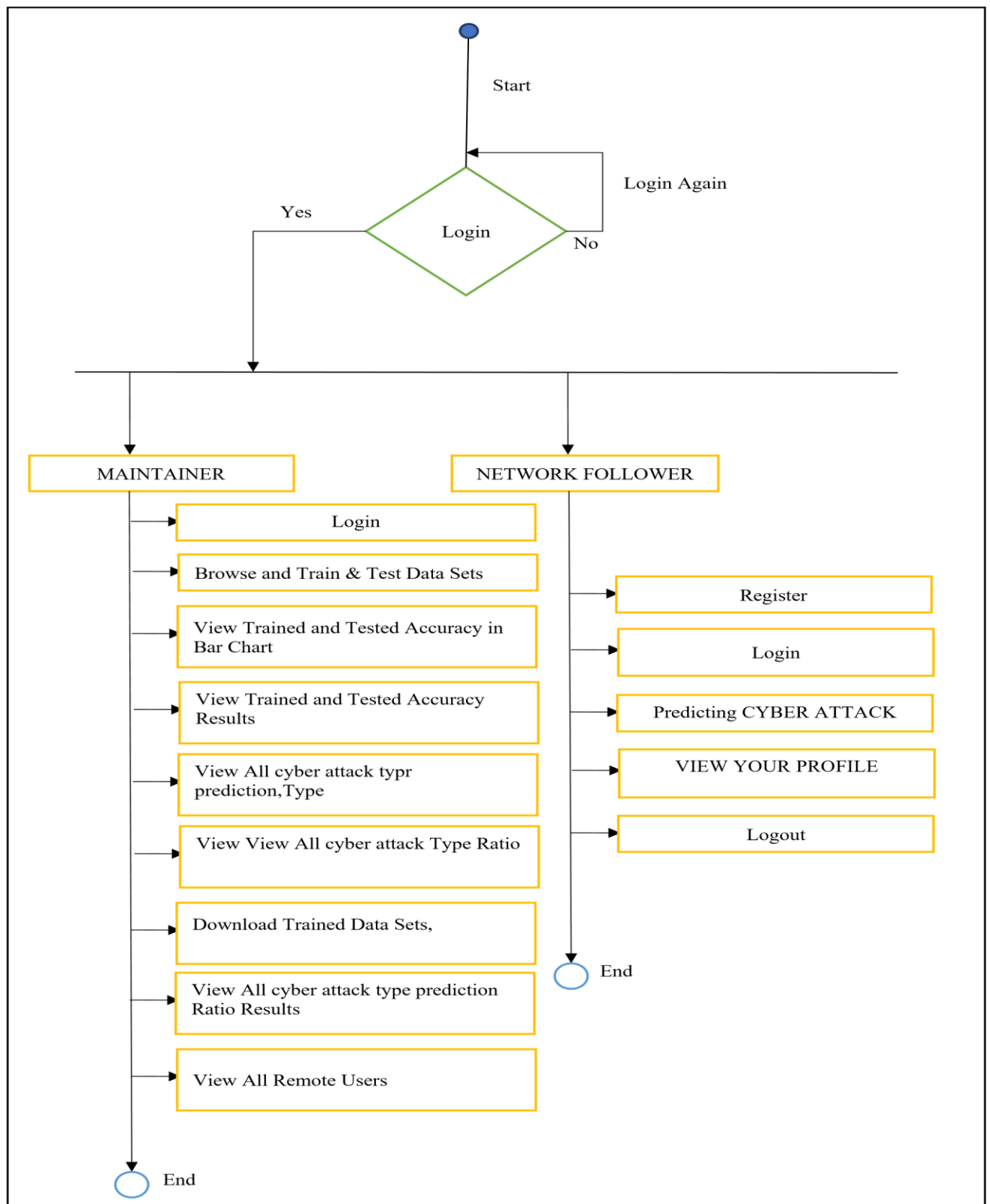


Fig 4.2.5: Activity Diagram

4.2.6 COMPONENT DIAGRAM

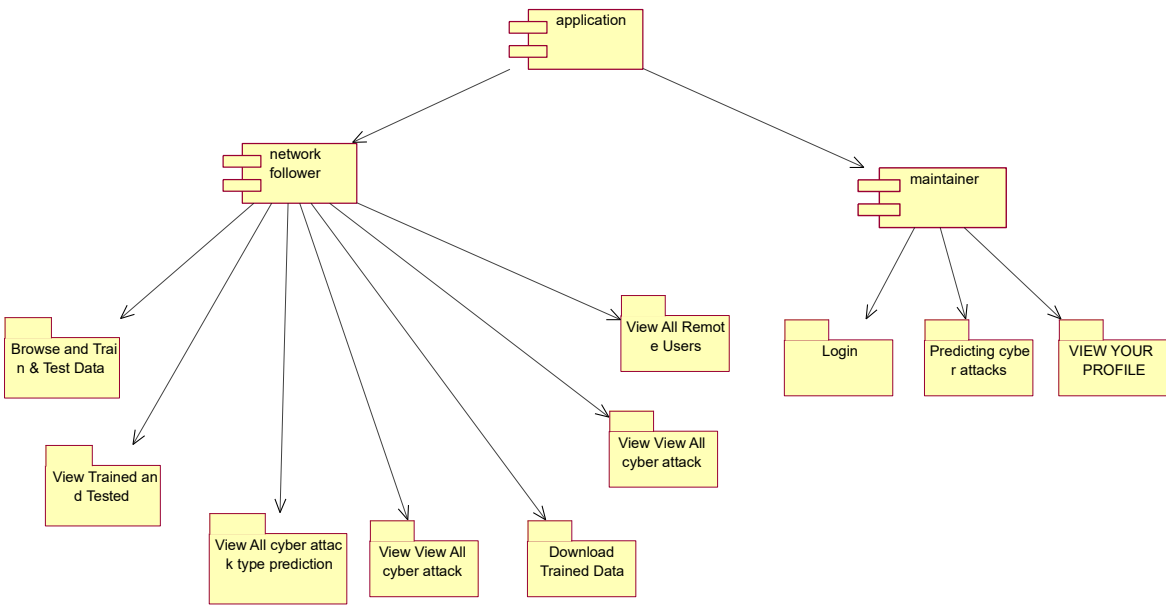


Fig 4.2.6: Component Diagram

4.2.7 DEPLOYMENT DIAGRAM

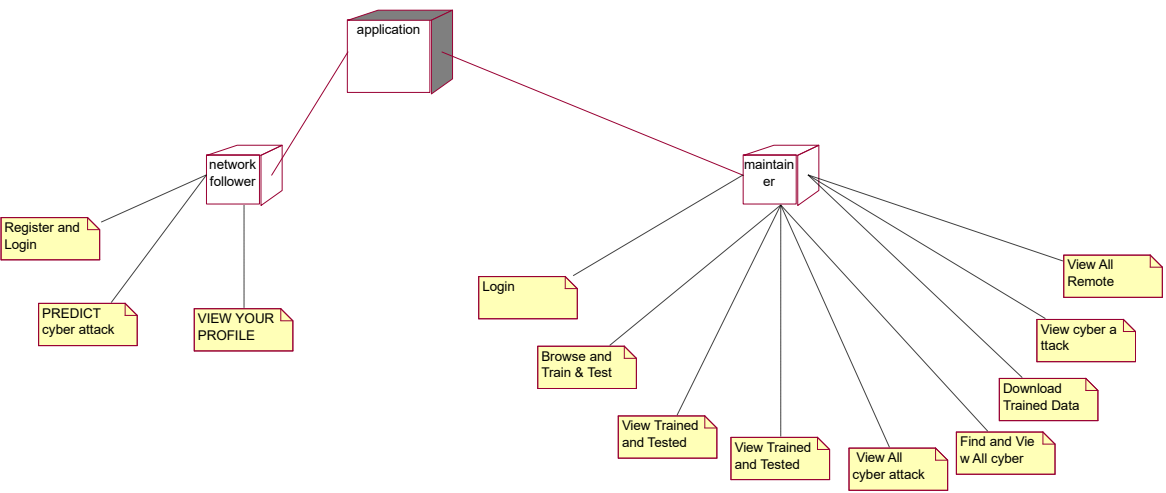


Fig 4.2.7: Deployment Diagram

4.2.8 ER DIAGRAM

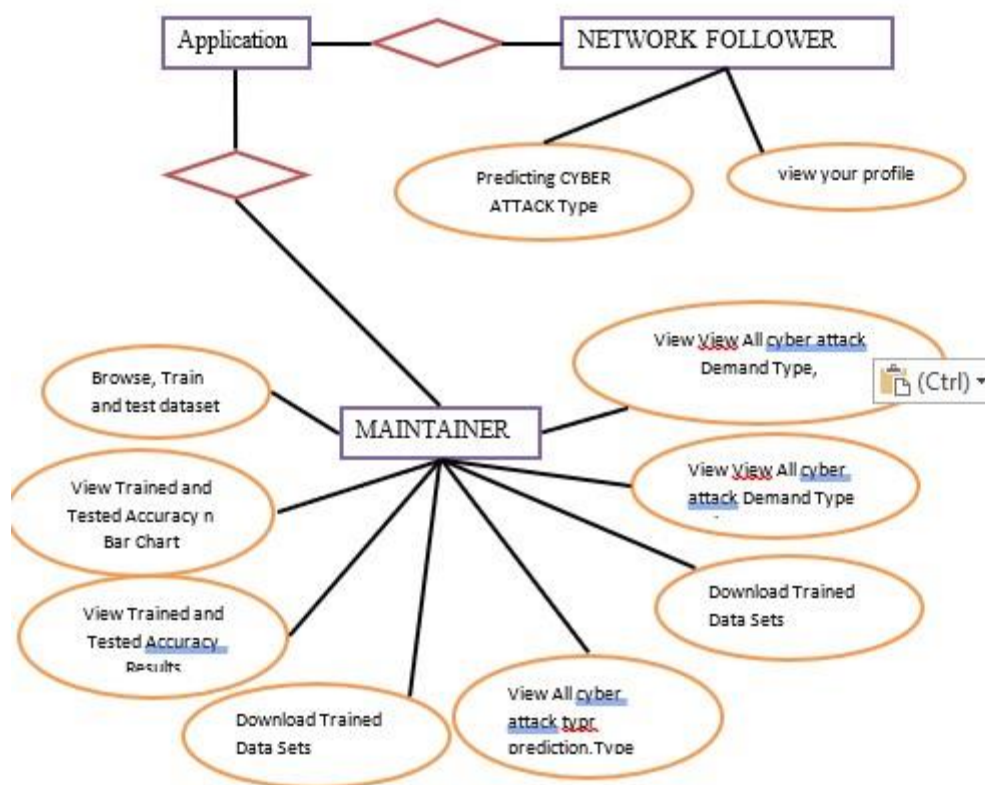


Fig 4.2.8:E-R Diagram

4.2.9 DATA DICTIONARY

Database: ARTIFICIAL_INTELLIGENCE

Table name: auth_group

Column	Data Type	Constraints	Description
id	Int(11)	Primary key	Unique Identifier
name	Varchar(1000)	Not null	name

Table Name: Auth_group_Permission

Table Name: auth_permission

Column	Data Type	Constraints	Description
id	Int(11)	Primary key	Unique Identifier
Group id	Int(11)	Primary Key	Unique Identifier
Permission_id	Int(11)	Primary Key	Unique Identifier
Column	Data Type	Constraints	Description

id	Int(11)	Primary key	Unique Identifier
Name	Int(255)	Not Null	name
Content_type_id	Int(11)	Primary Key	Unique Identifier
Code name	Int(100)	Not Null	name

Table Name: auth_User

Column	Data Type	Constraints	Description
id	Int(11)	Primary key	Unique Identifier
Password	varchar(128)	Not Null	password
Last_Login	Datetime(6)	Not Null	Last login
Is_superuser	tinyint(1)	Not Null	Name
username	Varchar(150)	Not Null	Username
lastname	Varchar(30)	Not Null	Lastname
email	Varchar(150)	Not Null	Email id
Date_joined	Datetime(6)	Not Null	Date and time

CHAPTER-5

IMPLEMENTATION

5.1 MODULES

- Maintainer
- Network Follower

5.1.1 MODULE DESCRIPTION

MAINTAINER

In this module, the Service Provider has to login by using valid user name and password. After login successful he can do some operations such as Browse and Train & Test Data Sets, View Trained and Tested Datasets Accuracy in Bar Chart, View Trained and Tested Datasets Accuracy Results, View Prediction Of Cyber Attack Type, View Prediction Of Cyber Attack Type Ratio, Download Predicted Data Sets, View Cyber Attack Type Ratio Results, View All Remote Users.

View and Authorize Users

In this module, the admin can view the list of users who all registered. In this, the admin can view the user's details such as, user name, email, address and admin authorizes the users.

NETWORK FOLLOWER

In this module, there are n numbers of users are present. User should register before doing any operations. Once user registers, their details will be stored to the database. After registration successful, he has to login by using authorized user name and password. Once Login is successful user will do some operations like REGISTER AND LOGIN, PREDICT CYBER ATTACK TYPE, VIEW YOUR PROFILE.

5.2 TECHNOLOGY

5.2.1 PYTHON

Python is a high-level, interpreted, interactive and object-oriented scripting language. Python is designed to be highly readable. Python is Interpreted, Interactive, Object-Oriented and is a Beginner's Language

HISTORY OF PYTHON

Python was developed by Guido van Rossum in the late eighties and early nineties at the National Research Institute for Mathematics and Computer Science in the Netherlands. Python is derived from many other languages, including ABC, Modula-3, C, C++, Algol-68, SmallTalk, and Unix shell and other scripting languages. Python is copyrighted. Like Perl, Python source code is now available under the GNU General Public License (GPL).

.

PYTHON FEATURES

Python's features include: Easy-to-learn

- Easy-to-read.
- Easy-to-maintain
- A broad standard library
- Interactive Mode
- Portable
- Extendable
- Databases
- GUI Programming
- Scalable.

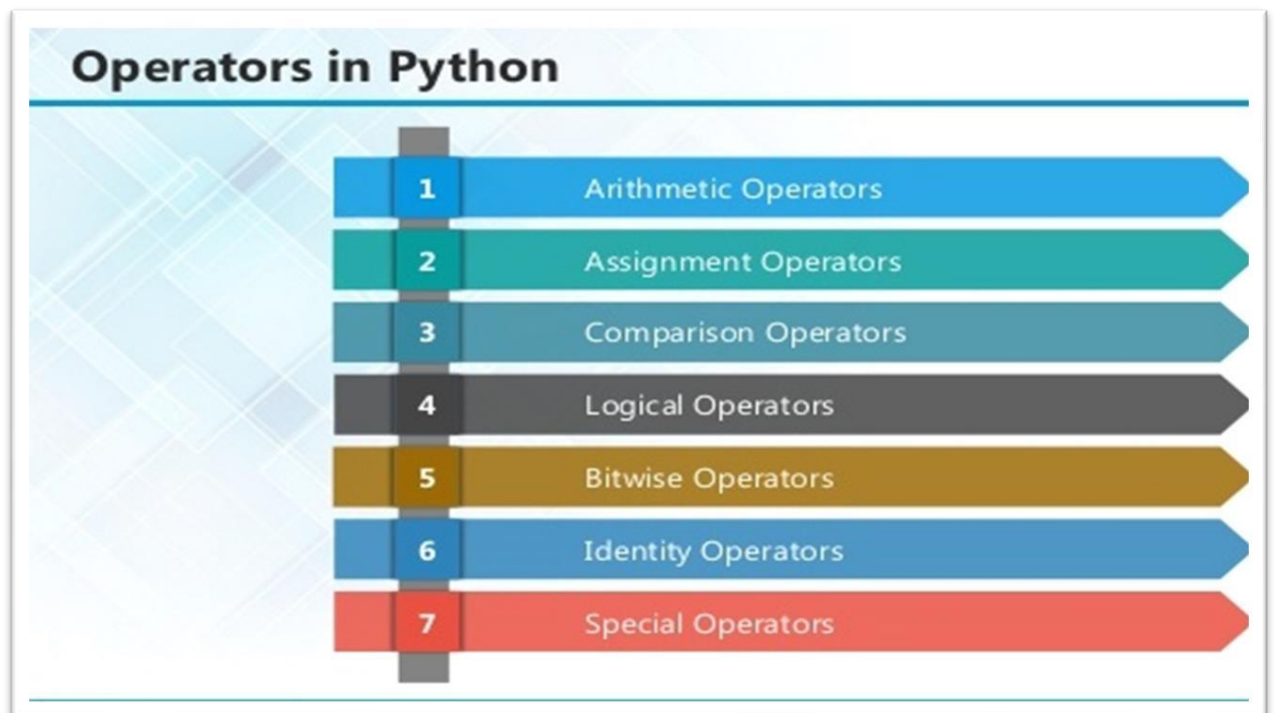


Fig 5.2.1 Operators in Python

ARITHMETIC OPERATORS

Operator	Description	Example
+ Addition	Adds values on either side of the operator.	$a + b = 30$
- Subtraction	Subtracts right hand operand from left hand operand.	$a - b = 10$
*Multiplication	Multiplies values on either side of the operator	$a * b = 200$
/ Division	Divides left hand operand by right hand operand	$b / a = 2$
% Modulus	Divides left hand operand by right hand operand and returns remainder	$b \% a = 0$
** Exponent	Performs exponential (power) calculation on operators	$a ** b = 10$ to the power 20
//	Floor Division - The division of operands where the result is the quotient in which the digits after the decimal point are removed.	$9 // 2 = 4$

ASSIGNMENT OPERATOR

Operator	Description	Example
=	Assigns values from right side operands to left side operand	$c = a + b$ assigns value of $a + b$ into c
+= Add AND	It adds right operand to the left operand and assign the result to left operand	$c += a$ is equivalent to $c = c + a$

<code>--</code> Subtract AND	It subtracts right operand from the left operand and assign the result to left operand	<code>c -= a</code> is equivalent to <code>c = c - a</code>
<code>*=</code> Multiply AND	It multiplies right operand with the left operand and assign the result to left operand	<code>c *= a</code> is equivalent to <code>c = c * a</code>
<code>/=</code> Divide AND	It divides left operand with the right operand and assign the result to left operand	<code>c /= a</code> is equivalent to <code>c = c / a</code> <code>c /= a</code> is equivalent to <code>c = c / a</code>

<code>%=</code> Modulus AND	It takes modulus using two operands and assign the result to left operand	<code>c %= a</code> is equivalent to <code>c = c % a</code>
<code>**=</code> Exponent AND	Performs exponential (power) calculation on operators and assign value to the left operand	<code>c **= a</code> is equivalent to <code>c = c ** a</code>
<code>//=</code> Floor Division	It performs floor division on operators and assign value to the left operand	<code>c //= a</code> is equivalent to <code>c = c // a</code>

IDENTITY OPERATOR

Operator	Description	Example
<code>is</code>	Evaluates to true if the variables on either side of the operator point to the same object and false otherwise.	<code>x is y</code> , here is results in 1 if <code>id(x)</code> equals <code>id(y)</code> .
<code>is not</code>	Evaluates to false if the variables on either side of the operator point to the same object and true otherwise.	<code>x is not y</code> , here is not results in 1 if <code>id(x)</code> is not equal to <code>id(y)</code>

COMPARISON OPERATOR

Operator	Description	Example
& Binary AND	Operator copies a bit to the result if it exists in both operands	(a & b) (means 0000 1100)
Binary OR	It copies a bit if it exists in either operand.	(a b) = 61 (means 0011 1101)
^ Binary XOR	It copies the bit if it is set in one operand but not both.	(a ^ b) = 49 (means 0011 0001)
~ Binary Ones Complement	It is unary and has the effect of 'flipping' bits.	(~a) = -61 (means 1100 0011)
<< Binary Left Shift	The left operands value is moved left by the number of bits specified by the right operand.	a << 2 = 240 (means 1111 0000)
>> Binary Right Shift	The left operands value is moved right by the number of bits specified by the right operand.	a >> 2 = 15 (means 0000 1111)

LOGICAL OPERATOR

Operator	Description	Example
and Logical AND	If both the operands are true then condition becomes true.	(a and b) is true.
or Logical OR	If any of the two operands are non-zero then condition becomes true.	(a or b) is true.
not Logical NOT	Used to reverse the logical state of its operand.	Not(a and b) is false.

Python Operators Precedence

Operator	Description
**	Exponentiation (raise to the power)
~ + -	Complement, unary plus and minus (method names for the last two are +@ and -@)
* / % //	Multiply, divide, modulo and floor division
+ -	Addition and subtraction
>> <<	Right and left bitwise shift
&	Bitwise 'AND'
^	Bitwise exclusive 'OR' and regular 'OR'
<= < > >=	Comparison operators
<> == !=	Equality operators
= %= /= //= -= += *= **=	Assignment operators
is is not	Identity operators

LIST

The list is a most versatile data type available in Python which can be written as a list of commaseparated values (items) between square brackets. Important thing about a list is that items in a list need not be of the same type.

```
list1 = ['physics', 'chemistry', 1997, 2000]; list2 =  
[1, 2, 3, 4, 5 ]; list3 = ["a", "b", "c", "d"]
```

Built-in List Functions & Methods:

Python includes the following list functions –

SN	Function with Description
1	<u>mp(list1, list2)</u> Compares elements of both lists.

2	<u>len(list)</u> Gives the total length of the list.
3	<u>max(list)</u> Returns item from the list with max value.
4	<u>min(list)</u> Returns item from the list with min value.
5	<u>list(seq)</u> Converts a tuple into list.

TUPLES

A tuple is a sequence of immutable Python objects. Tuples are sequences, just like lists. The differences between tuples and lists are, the tuples cannot be changed unlike lists and tuples use parentheses, whereas lists use square brackets.

- **Accessing Values in Tuples:**

To access values in tuple, use the square brackets for slicing along with the index or indices to obtain value available at that index. For example –

```
tup1 = ('physics', 'chemistry', 1997, 2000); tup2
= (1, 2, 3, 4, 5, 6, 7 ); print "tup1[0]: ", tup1[0]
print "tup2[1:5]: ", tup2[1:5]
```

When the code is executed, it produces the following result –

```
tup1[0]: physics tup2[1:5]: [2, 3,
4, 5]
```

Updating Tuples:

Tuples are immutable which means you cannot update or change the values of tuple elements. We are able to take portions of existing tuples to create new tuples as the following example demonstrates –

```
tup1 = (12, 34.56); tup2 =
('abc', 'xyz'); tup3 = tup1 +
tup2; print tup3
```

When the above code is executed, it produces the following result –

```
(12, 34.56, 'abc', 'xyz')
```

Delete Tuple Elements

Removing individual tuple elements is not possible. There is, of course, nothing wrong with putting together another tuple with the undesired elements discarded.

To explicitly remove an entire tuple, just use the **del** statement. For example:

```
tup = ('physics', 'chemistry', 1997,  
2000); print tup  
del tup  
print "After deleting tup : " print  
tup
```

DICTIONARY

Each key is separated from its value by a colon (:), the items are separated by commas, and the whole thing is enclosed in curly braces. An empty dictionary without any items is written with just two curly braces, like this: {}.

Keys are unique within a dictionary while values may not be. The values of a dictionary can be of any type, but the keys must be of an immutable data type such as strings, numbers, or tuples.

Accessing Values in Dictionary: To access dictionary elements, you can use the familiar square brackets along with the key to obtain its value. Following is a simple example –

```
dict = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}  
  
print "dict['Name']: ", dict['Name']  
print "dict['Age']: ", dict['Age']
```

Result –

```
dict['Name']: Zara dict['Age']: 7
```

Built-in Dictionary Functions & Methods –

Python includes the following dictionary functions –

SN	Function with Description
1	<u>cmp(dict1, dict2)</u> Compares elements of both dict.
2	<u>len(dict)</u> Gives the total length of the dictionary. This would be equal to the number of items in the dictionary.
3	<u>str(dict)</u> Produces a printable string representation of a dictionary
4	<u>type(variable)</u> Returns the type of the passed variable. If passed variable is dictionary, then it would return a dictionary type.

Updating Dictionary

We can update a dictionary by adding a new entry or a key-value pair, modifying an existing entry, or deleting an existing entry as shown below in the simple example – `dict = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}`

```
dict['Age'] = 8; # update existing entry dict['School'] = "DPS School"; # Add new entry
print "dict['Age']: ", dict['Age'] print "dict['School']: ", dict['School']
```

Result –

```
dict['Age']: 8
dict['School']: DPS School
```

Delete Dictionary Elements

We can either remove individual dictionary elements or clear the entire contents of a dictionary. You can also delete entire dictionary in a single operation.

To explicitly remove an entire dictionary, just use the **del** statement. Following is a simple example –

Built-in Dictionary Functions & Methods –

Python includes the following dictionary functions –

SN	Function with Description
1	<u>cmp(dict1, dict2)</u> Compares elements of both dict.
2	<u>len(dict)</u> Gives the total length of the dictionary. This would be equal to the number of items in the dictionary.
3	<u>str(dict)</u> Produces a printable string representation of a dictionary
4	<u>type(variable)</u> Returns the type of the passed variable. If passed variable is dictionary, then it would return a dictionary type.

Defining a Function

Simple rules to define a function in Python.

- Function blocks begin with the keyword `def` followed by the function name and parentheses `(( ))`.
- Any input parameters or arguments should be placed within these parentheses. You can also define parameters inside these parentheses.
- The first statement of a function can be an optional statement - the documentation string of the function or *docstring*.
- The code block within every function starts with a colon `(:)` and is indented.
- The statement `return [expression]` exits a function, optionally passing back an expression to the caller. A return statement with no arguments is the same as `return None`.

Calling a Function

Defining a function only gives it a name, specifies the parameters that are to be included in the function and structures the blocks of code. Once the basic structure of a function is finalized, you can execute it by calling it from another function or directly from the Python prompt. Following is the example to call `printme()` function –

Function Arguments

You can call a function by using the following types of formal arguments:

- Required arguments
- Keyword arguments
- Default arguments
- Variable-length arguments

Scope of Variables

All variables in a program may not be accessible at all locations in that program. This depends on where you have declared a variable. The scope of a variable determines the portion of the program where you can access a particular identifier. There are two basic scopes of variables in Python –

Global variables

Local variables

Global vs. Local variables

Variables that are defined inside a function body have a local scope, and those defined outside have a global scope. This means that local variables can be accessed only inside the function in which they are declared, whereas global variables can be accessed throughout the program body by all functions. When you call a function, the variables declared inside it are brought into scope.

A module allows you to logically organize your Python code. Grouping related code into a module makes the code easier to understand and use. A module is a Python object with arbitrarily named attributes that you can bind and reference. Simply, a module is a file consisting of Python code. A module can define functions, classes and variables. A module can also include runnable code.

Example:

The Python code for a module named *aname* normally resides in a file named *aname.py*. Here's an example of a simple module, support.py

```
def print_func( par ): print
"Hello : ", par return
```

The import Statement

The import has the following syntax:

```
import module1[, module2[,... moduleN]
```

When the interpreter encounters an import statement, it imports the module if the module is present in the search path. A search path is a list of directories that the interpreter searches before importing a module. For example, to import the module support.py, you need to put the following command at

the top of the script – A module is loaded only once, regardless of the number of times it is imported. This prevents the module execution from happening over and over again if multiple imports occur.

Packages in Python

A package is a hierarchical file directory structure that defines a single Python application environment that consists of modules and sub packages and sub-sub packages.

Consider a file Pots.py available in Phone directory. This file has following line of source code –

```
def Pots(): print "I'm Pots  
Phone"
```

Reading Keyboard Input

Python provides two built-in functions to read a line of text from standard input, which by default comes from the keyboard. These functions are –

- raw_input
- input

The raw_input([prompt]) function reads one line from standard input and returns it as a string (removing the trailing newline).

The input([prompt]) function is equivalent to raw_input, except that it assumes the input is a valid Python expression and returns the evaluated result to you.

Opening and Closing Files

Python provides basic functions and methods necessary to manipulate files by default. You can do most of the file manipulation using a file object.

Here are parameter details:

- file_name: The file_name argument is a string value that contains the name of the file that you want to access.

- **access_mode:** The access_mode determines the mode in which the file has to be opened, i.e., read, write, append, etc. A complete list of possible values is given below in the table. This is optional parameter and the default file access mode is read (r).
- **buffering:** If the buffering value is set to 0, no buffering takes place. If the buffering value is 1, line buffering is performed while accessing a file. If you specify the buffering value as an integer greater than 1, then buffering action is performed with the indicated buffer size.
If negative, the buffer size is the system default(default behavior).

Here is a list of the different modes of opening a file –

Modes	Description
r	Opens a file for reading only.
rb	Opens a file for reading only in binary format
r+	Opens a file for both reading and writing.
rb+	Opens a file for both reading and writing in binary format.
w	Opens a file for writing only. Overwrites the file if the file exists.
wb	Opens a file for writing only in binary format. Overwrites the file if the file exists.
w+	Opens a file for both writing and reading. Overwrites the existing file if the file exists.
wb+	Opens a file for both writing and reading in binary format. Overwrites the existing file if the file exists.
a	Opens a file for appending. The file pointer is at the end of the file if the file exists
ab	Opens a file for appending in binary format. The file pointer is at the end of the file if the file exists.
a+	Opens a file for both appending and reading. The file pointer is at the end of the file if the file exists

The file Object Attributes

Once a file is opened and you have one file object, you can get various information related to that file.

Here is a list of all attributes related to file object:

Example

```
# Open a file fo = open("foo.txt", "wb")
print "Name of the file: ", fo.name
print "Closed or not : ", fo.closed
print "Opening mode : ", fo.mode
print "Softspace flag : ", fo.softspace
This produces the following result –
```

```
Name of the file: foo.txt
Closed or not : False
Opening mode : wb
Softspace flag : 0
```

The close() Method

The close() method of a *file* object flushes any unwritten information and closes the file object, after which no more writing can be done.

Attribute	Description
file.closed	Returns true if file is closed, false otherwise.
file.mode	Returns access mode with which file was opened.
file.name	Returns name of the file.
file.softspace	Returns false if space explicitly required with print, true otherwise.

Syntax

```
fileObject.close();
```

Reading and Writing Files

The file object provides a set of access methods to make our lives easier. We would see how to use read() and write() methods to read and write files.

The write() Method

The write() method writes any string to an open file. It is important to note that Python strings can have binary data and not just text. The write() method does not add a newline character ('\n') to the end of the string

Syntax

```
fileObject.write(string);
```

Here, passed parameter is the content to be written into the opened file. The above method would create foo.txt file and would write given content in that file and finally it would close that file.

The read() Method

The read() method reads a string from an open file. It is important to note that Python strings can have binary data. apart from text data.

Syntax

```
fileObject.read([count]);
```

File Positions

The tell() method tells you the current position within the file; in other words, the next read or write will occur at that many bytes from the beginning of the file. The seek(offset[, from]) method changes the current file position. The offset argument indicates the number of bytes to be moved. The from argument specifies the reference position from where the bytes are to be moved. If from is set to 0, it means use the beginning of the file as the reference position and 1 means use the current position as the reference position and if it is set to 2 then the end of the file would be taken as the reference position.

Renaming and Deleting Files

Python **os** module provides methods that help you perform file-processing operations, such as renaming and deleting files.

To use this module you need to import it first and then you can call any related functions.

The rename() Method

The rename() method takes two arguments, the current filename and the new filename.

Syntax

```
os.rename(current_file_name, new_file_name)
```

You can use the remove() method to delete files by supplying the name of the file to be deleted as the argument.

Syntax

```
os.remove(file_name)
```

Directories in Python

All file are contained within various directories, and Python has no problem handling these too.

The **os** module has several methods that help you create, remove, and change directories.

The mkdir() Method

You can use the mkdir() method of the **os** module to create directories in the current directory. You need to supply an argument to this method which contains the name of the directory to be created.

Syntax

```
os.mkdir("newdir")
```

You can use the chdir() method to change the current directory. The chdir() method takes an argument, which is the name of the directory that you want to make the current directory.

Syntax

```
os.chdir("newdir")
```

The getcwd() Method

The getcwd() method displays the current working directory.

Syntax

```
os.getcwd()
```

The rmdir() Method

The rmdir() method deletes the directory, which is passed as an argument in the method.

Before removing a directory, all the contents in it should be removed.

Syntax:

```
os.rmdir('dirname')
```

File & Directory Related Methods

There are three important sources, which provide a wide range of utility methods to handle and manipulate files & directories on Windows and Unix operating systems. They are as follows –

- File Object Methods: The file object provides functions to manipulate files.
- OS Object Methods: This provides methods to process files as well as directories. –
- **Exception Handling**: This would be covered in this tutorial. Here is a list standard Exceptions available in Python: Standard Exceptions.
- **Assertions**: This would be covered in Assertions in Python

List of Standard Exceptions –

EXCEPTION NAME	DESCRIPTION
Exception	Base class for all exceptions
StopIteration	Raised when the next() method of an iterator does not point to any object.
SystemExit	Raised by the sys.exit() function.
StandardError	Base class for all built-in exceptions except StopIteration and SystemExit.
ArithmeticError	Base class for all errors that occur for numeric calculation.
OverflowError	Raised when a calculation exceeds maximum limit for a numeric type.
FloatingPointError	Raised when a floating point calculation fails.
ZeroDivisionError	Raised when division or modulo by zero takes place for all numeric types.
AssertionError	Raised in case of failure of the Assert statement.
AttributeError	Raised in case of failure of attribute reference or assignment.
EOFError	Raised when there is no input from either the raw_input() or input() function and the end of file is reached.
KeyboardInterrupt	Raised when the user interrupts program execution, usually by pressing Ctrl+c.
LookupError	Base class for all lookup errors.
IndexError	Raised when an index is not found in a sequence.
KeyError	Raised when the specified key is not found in the dictionary.
NameError	Raised when an identifier is not found in the local or global namespace.

UnboundLocalError	Raised when trying to access a local variable in a function or method but no value has been assigned to it.
EnvironmentError	Base class for all exceptions that occur outside the Python environment.
IOError	Raised when an input/ output operation fails, such as the print statement or the open() function when trying to open a file that does not exist.
IOError	Raised for operating system-related errors.
SyntaxError	Raised when there is an error in Python syntax.
IndentationError	Raised when indentation is not specified properly.
SystemError	Raised when the interpreter finds an internal problem, but when this error is encountered the Python interpreter does not exit.
SystemExit	Raised when Python interpreter is quit by using the sys.exit() function. If not handled in the code, causes the interpreter to exit.
TypeError	Raised when an operation or function is attempted that is invalid for the specified data type.
ValueError	Raised when the built-in function for a data type has the valid type of arguments, but the arguments have invalid values specified.
RuntimeError	Raised when a generated error does not fall into any category.
NotImplementedError	Raised when an abstract method that needs to be implemented in an inherited class is not actually implemented.

What is Exception?

An exception is an event, which occurs during the execution of a program that disrupts the normal flow of the program's instructions. In general, when a Python script encounters a situation that it cannot cope with, it raises an exception. An exception is a Python object that represents an error. When a Python script raises an exception, it must either handle the exception immediately otherwise it terminates and quits.

Handling an exception

If you have some suspicious code that may raise an exception, you can defend your program by placing the suspicious code in a **try:** block. After the try: block, include an **except:** statement, followed by a block of code which handles the problem as elegantly as possible.

The Python standard for database interfaces is the Python DB-API. Most Python database interfaces adhere to this standard.

You can choose the right database for your application. Python Database API supports a wide range of database servers such as –

- GadFly
- mSQL
- MySQL
- PostgreSQL
- Microsoft SQL Server 2000
- Informix
- Interbase
- Oracle
- Sybase

The DB API provides a minimal standard for working with databases using Python structures and syntax wherever possible. This API includes the following:

- Importing the API module.
- Acquiring a connection with the database.
- Issuing SQL statements and stored procedures.
- Closing the connection

5.3 SYSTEM TESTING

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub assemblies, assemblies and/or a finished product. It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of test. Each test type addresses a specific testing requirement.

TYPES OF TESTS

Unit testing

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application .it is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

Integration testing

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfaction, as shown by successfully unit testing, the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components.

Functional test

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

Functional testing is centered on the following items:

- Valid Input : identified classes of valid input must be accepted.
- Invalid Input : identified classes of invalid input must be rejected.
- Functions : identified functions must be exercised.
- Output : identified classes of application outputs must be exercised.

Systems/Procedures: interfacing systems or procedures must be invoked.

Organization and preparation of functional tests is focused on requirements, key functions, or special test cases. In addition, systematic coverage pertaining to identify Business process flows; data fields, predefined processes, and successive processes must be considered for testing. Before functional testing is complete, additional tests are identified and the effective value of current tests is determined. **System Test**

System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. An example of system testing is the configuration oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

White Box Testing

White Box Testing is a testing in which in which the software tester has knowledge of the inner workings, structure and language of the software, or at least its purpose. It is purpose. It is used to test areas that cannot be reached from a black box level.

Black Box Testing

Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests, as most other kinds of tests, must be written from a definitive source document, such as specification or requirements document, such as specification or requirements document. It is a testing in which the software under test is treated, as a black box .you cannot “see” into it. The test provides inputs and responds to outputs without considering how the software works.

Unit Testing:

Unit testing is usually conducted as part of a combined code and unit test phase of the software lifecycle, although it is not uncommon for coding and unit testing to be conducted as two distinct phases.

Test strategy and approach

Field testing will be performed manually and functional tests will be written in detail.

Test objectives

- All field entries must work properly.
- Pages must be activated from the identified link.
- The entry screen, messages and responses must not be delayed.

Features to be tested

- Verify that the entries are of the correct format
- No duplicate entries should be allowed
- All links should take the user to the correct page.

Integration Testing

Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects.

The task of the integration test is to check that components or software applications, e.g. components in a software system or – one step up – software applications at the company level – interact without error.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

Acceptance Testing

User Acceptance Testing is a critical phase of any project and requires significant participation by the end user. It also ensures that the system meets the functional requirements.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

The following are the **Testing Methodologies**:

- Unit Testing.
- Integration Testing.
- User Acceptance Testing.
- Output Testing.
- Validation Testing.

Unit Testing

Unit testing focuses verification effort on the smallest unit of Software design that is the module. Unit testing exercises specific paths in a module's control structure to ensure complete coverage and maximum error detection. This test focuses on each module individually, ensuring that it functions properly as a unit. Hence, the naming is Unit Testing. During this testing, each module is tested individually and the module interfaces are verified for the consistency with design specification. All important processing path are tested for the expected results. All error handling paths are also tested.

Integration Testing

Integration testing addresses the issues associated with the dual problems of verification and program construction. After the software has been integrated a set of high order tests are conducted. The main objective in this testing process is to take unit tested modules and builds a program structure that has been dictated by design.

The following are the types of Integration Testing:

1)Top-Down Integration

This method is an incremental approach to the construction of program structure. Modules are integrated by moving downward through the control hierarchy, beginning with the main program module. The module subordinates to the main program module are incorporated into the structure in either a depth first or breadth first manner. In this method, the software is tested from main module and individual stubs are replaced when the test proceeds downwards.

Bottom-up Integration

This method begins the construction and testing with the modules at the lowest level in the program structure. Since the modules are integrated from the bottom up, processing required for modules subordinate to a given level is always available and the need for stubs is eliminated. The bottom up integration strategy may be implemented with the following steps:

- The low-level modules are combined into clusters into clusters that perform a specific Software sub-function.
- A driver (i.e.) the control program for testing is written to coordinate test case input and output.
- The cluster is tested.
- Drivers are removed and clusters are combined moving upward in the program structure

The bottom up approaches tests each module individually and then each module is module is integrated with a main module and tested for functionality.

TESTING STRATEGY :

A strategy for system testing integrates system test cases and design techniques into a well planned series of steps that results in the successful construction of software. The testing strategy must cooperate test planning, test case design, test execution, and the resultant data collection and evaluation .A strategy for software testing must accommodate low-level tests that are necessary to verify that a small source code segment has been correctly implemented as well as high level tests that validate major system functions against user requirements.

Software testing is a critical element of software quality assurance and represents the ultimate review of specification design and coding. Testing represents an interesting anomaly for the software. Thus, a series of testing are performed for the proposed system before the system is ready for user acceptance testing.

SYSTEM TESTING:

Software once validated must be combined with other system elements (e.g. Hardware, people, database). System testing verifies that all the elements are proper and that overall system function performance is achieved. It also tests to find discrepancies between the system and its original objective, current specifications and system documentation.

UNIT TESTING:

In unit testing different are modules are tested against the specifications produced during the design for the modules. Unit testing is essential for verification of the code produced during the coding phase, and hence the goals to test the internal logic of the modules. Using the detailed design description as a guide, important Conrail paths are tested to uncover errors within the boundary of the modules. This testing is carried out during the programming stage itself. In this type of testing step, each module was found to be working satisfactorily as regards to the expected output from the module.

TEST CASSES

Test case1:

Test case for Login form:

FUNCTION:	LOGIN
EXPECTED RESULTS:	Should Validate the user and check his existence in database
ACTUAL RESULTS:	Validate the user and checking the user against the database
LOW PRIORITY	No
HIGH PRIORITY	Yes

Test case2:

Test case for User Registration form:

FUNCTION	REMOTE USER REGISTRATION
EXPECTED RESULTS:	Should check if all the fields are filled by the user and saving the user to database.
ACTUAL RESULTS:	Checking whether all the fields are filled by user or not through validations and saving user.
LOW PRIORITY	No
HIGH PRIORITY	Yes

Test case3:

Test case for Change Password: When the old password does not match with the new password , then this results in displaying an error message as “ OLD PASSWORD DOES NOT MATCH WITH THE NEW PASSWORD”.

Test case 4:

Test case for Forget Password:

When a user forgets his password he is asked to enter Login name, ZIP code, Mobile number. If these are matched with the already stored ones then user will get his Original password.

Module	Functionality	Test Case	Expected Results	Actual Results	Result	Priority
er	Login Usecase	Navigate To Www.Sample.Com Click On Submit Button Without Entering Username and Password	A Validation Should Be As Below “Please Enter Valid Username & Password”	A Validation Has Been Populated As Expected	Pass	High

		Navigate ToWww.Sample.Com Click On Submit Button With Out Filling Password And With Valid Username.Test Us ername Field .Navigate To Www.Sample.Com Enter Both Username And Password Wrong And Hit Enter	A Validation Should Be As Below “Please Enter Valid Password Or Password Field Can Not Be Empty “ A Validation Shown As Below “The Username Entered Is Wrong”	A Validation Is Shown As Expected A Validation Is Shown As Expected	Pass	High
		Navigate To Www.Sample.Com Enter Validate Username And Password And Click On Submit	Validate Username And Password In DataBase And Once If They Correct Then Show Main Page	Main Page/ Home Page Has Been Displayed	Pass	High

CHAPTER-6

RESULTS&DISCUSSIONS

The Influence of Artificial Intelligence on E-Governance and Cybersecurity in Smart Cities



FIG 6. 1 : HOME SCREEN

Artificial intelligence, cybersecurity, e-Governance, stakeholder involvement, machine learning, computer crime, smart cities, data privacy..

Login Using Your Account:

Admin

LOGIN

Are You New User !!! REGISTER

FIG 6. 2 : NETWORK FOLLOWER LOGIN AGE

Artificial intelligence, cybersecurity, e-Governance, stakeholder involvement, machine learning, computer crime, smart cities, data privacy..

REGISTER YOUR DETAILS HERE !!!

Enter Username	Admin	Enter Password	*****
Enter EMail Id	Enter Email	Enter Address	Enter Address
Enter Gender	----Select Gender ----	Enter Mobile Number	Enter Mobile Number
Enter Country Name	Enter Country Name	Enter State Name	Enter State Name
Enter City Name	Enter City Name		

REGISTER

FIG 6.3 : NETWORK FOLLOWER REGISTRATION PAGE



FIG 6.4 : MAINTAINER LOGIN PAGE

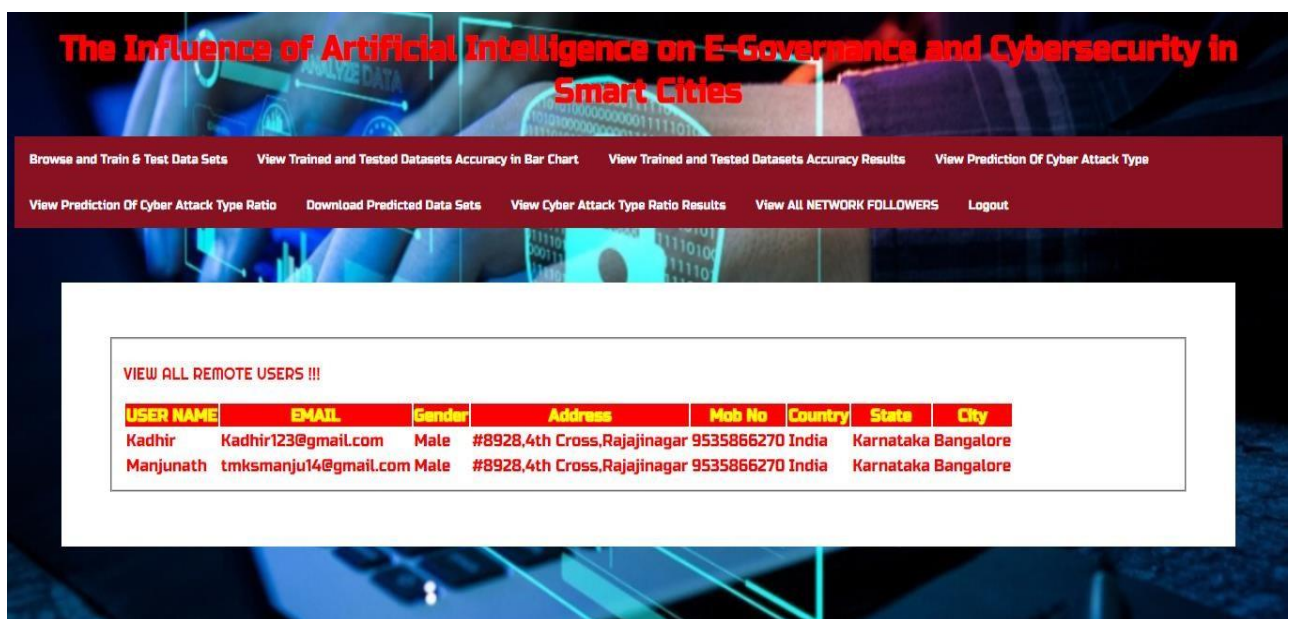


FIG 6.5 : MAINTAINER LOGIN HOME PAGE



FIG 6.6 : MAINTAINER OPERATION PAGE

CHAPTER-7

CONCLUSION

7.1 CONCLUSION

The current study examined artificial intelligence applications to overcome cyber security challenges. The research findings indicate that artificial intelligence is progressively converting into an indispensable technology to enhance information security performance. Individuals are not capable anymore of fully secure project-level cyber attacks, and artificial intelligence offers the desired analytics and threat intelligence that security practitioners might use to minimize the likelihood of an infringement and strengthen the security structure of an enterprise. Since more technologies computing in cyber security is the capacity to evaluate and eliminate risk faster. Several individuals are concerned about cybercriminals' capability to perform incredibly advanced cyber and technological attacks. Moreover, artificial intelligence can contribute to the detection and classification of hazards, the structuring of incident management, and the detection of cyber attacks before their occurrence. Consequently, despite potential negatives, artificial intelligence would contribute to the evolution of cyber security and support enterprises in establishing an enhanced security strategy.

This study further sought to investigate artificial intelligence and its ongoing development in offering e-government services and then highlight the need to accommodate strategies regarding cyber security for adopting innovative social and technical processes in government serving the community. The eventual objective of smart city governments is to establish and strengthen relationships with most stakeholders, as their involvement strengthens e-government efficacy which fortifies cyber security. Public services should be administered using innovative AI technologies and e-governance in convenient modes to eliminate the barriers between stakeholders and city governments, while state officials can still sustain the model for better support. While e-government is progressing, the citizens and those in authority or advocating mechatronics are lagging. That creates disparities in cyber security standards for something in the virtual environment, potentially turning performance into a much more difficult experience with several grooves to monitor. With an elevation in the initiatives identified in this research, stakeholders' involvement and awareness of e-governance and cyber security may rise, enabling benefits associated with the virtual environment.

7.2 FUTURE SCOPE

The future scope of the project titled "The Influence of Artificial Intelligence on E-Governance and Cybersecurity in Smart Cities" is vast and multifaceted. As smart cities evolve, the integration of AI can revolutionize e-governance by enhancing decision-making processes, improving public services, and fostering citizen engagement through predictive analytics and personalized services. AI-driven systems can streamline administrative tasks, optimize resource allocation, and reduce operational costs, leading to more efficient and responsive governance. In terms of cybersecurity, AI's role is crucial in proactively identifying and mitigating threats, managing vulnerabilities, and ensuring the privacy and integrity of citizens' data. Advanced AI algorithms can detect anomalies in real-time, automate incident responses, and adapt to emerging cyber threats, thereby strengthening the overall security framework of smart cities. Moreover, the continuous development of AI technologies can pave the way for innovative applications in areas such as traffic management, environmental monitoring, and emergency response, further enhancing the quality of urban life. As AI continues to advance, its influence on e-governance and cybersecurity will become increasingly significant, driving smart cities towards a more secure, efficient, and sustainable future.

REFERENCES

- [1] B. Alhayani, H. J. Mohammed, I. Z. Chaloob, and J. S. Ahmed, “Effectiveness of artificial intelligence techniques against cyber security risks apply of IT industry,” *Mater. Today, Proc.*, vol. 531, pp. 1–6, 2021, doi: [10.1016/j.matpr.2021.02.531](https://doi.org/10.1016/j.matpr.2021.02.531).
- [2] M. Komar, V. Kochan, L. Dubchak, A. Sachenko, V. Golovko, S. Bezobrazov, and I. Romanets, “High performance adaptive system for cyber attacks detection,” in *Proc. 9th IEEE Int. Conf. Intell. Data Acquisition Adv. Comput. Syst., Technol. Appl. (IDAACS)*, vol. 2, Sep. 2017, pp. 853–858.
- [3] M. D. Cavelty, *Cyber-Security and Threat Politics: US Efforts to Secure the Information Age*. Evanston, IL, USA: Routledge, 2007.
- [4] F. Fransen, A. Smulders, and R. Kerkdijk, “Cyber security information exchange to gain insight into the effects of cyber threats and incidents,” *Elektrotechnik Informationstechnik*, vol. 132, no. 2, pp. 106–112, Mar. 2015.
- [5] A. Corallo, M. Lazoi, M. Lezzi, and A. Luperto, “Cybersecurity awareness in the context of the industrial Internet of Things: A systematic literature review,” *Comput. Ind.*, vol. 137, May 2022, Art. no. 103614.
- [6] G. A. Weaver, B. Feddersen, L. Marla, D. Wei, A. Rose, and M. Van Moer, “Estimating economic losses from cyber-attacks on shipping ports: An optimization-based approach,” *Transp. Res. C, Emerg. Technol.*, vol. 137, Apr. 2022, Art. no. 103423.
- [7] M. Bada and J. R. C. Nurse, “The social and psychological impact of cyberattacks,” in *Emerging Cyber Threats and Cognitive Vulnerabilities*. Amsterdam, The Netherlands: Elsevier, 2020, pp. 73–92.
- [8] G. Allen and T. Chan, *Artificial Intelligence and National Security*. Cambridge, MA, USA: Belfer Center for Science and International Affairs, 2017.
- [9] Z. Zhang, H. Ning, F. Shi, F. Farha, Y. Xu, J. Xu, F. Zhang, and K.-K. R. Choo, “Artificial intelligence in cyber security: Research advances, challenges, and opportunities,” *Artif. Intell. Rev.*, vol. 55, pp. 1029–1053, Feb. 2022.
- [10] Z. I. Khisamova, I. R. Begishev, and E. L. Sidorenko, “Artificial intelligence and problems of ensuring cyber security,” *Int. J. Cyber Criminol.*, vol. 13, no. 2, pp. 564–577, 2019.